Intensity Transformation on Image

```
[2]
✓ 0s
import cv2
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['figure.autolayout'] = True
```

```
[ ]
print(np.e)
```

```
2.718281828459045
```

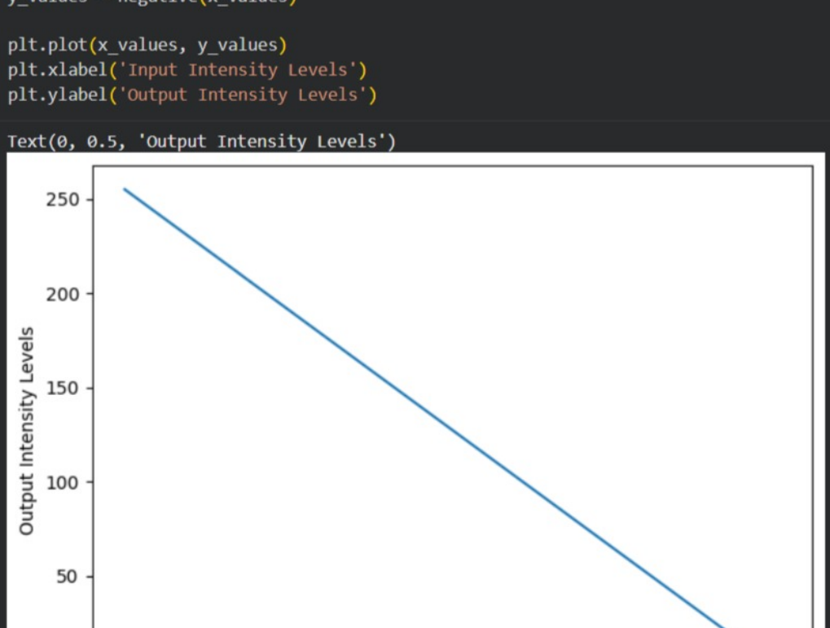
Image negatives

```
[ ]
img = cv2.imread('Fig0304(a)(breast_digital_Xray).tif')
```

```
def negative(r):
  s = 255 - r
  return s
```

```
x_values = np.linspace(0, 255, 500)
y_values = negative(x_values)

plt.plot(x_values, y_values)
plt.xlabel('Input Intensity Levels')
plt.ylabel('Output Intensity Levels')
```

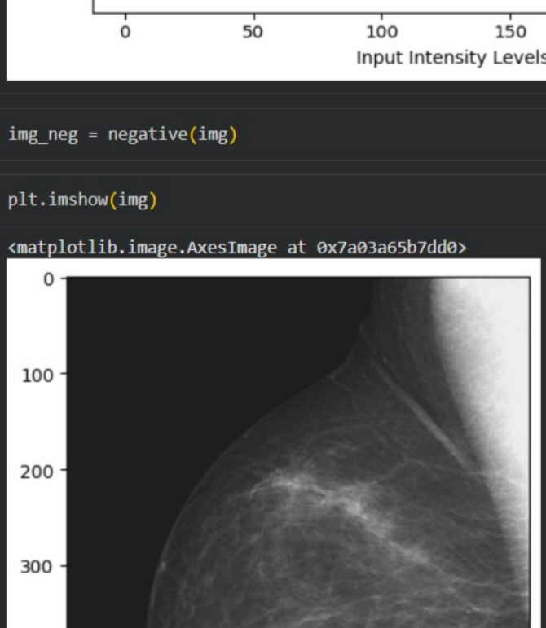
```
Text(0, 0.5, 'Output Intensity Levels')
```



```
img_neg = negative(img)
```

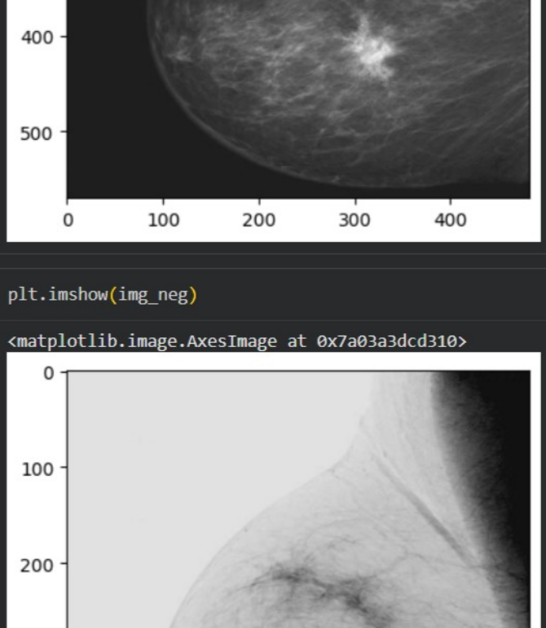
```
plt.imshow(img)
```

```
<matplotlib.image.AxesImage at 0x7a03a65b7dd0>
```



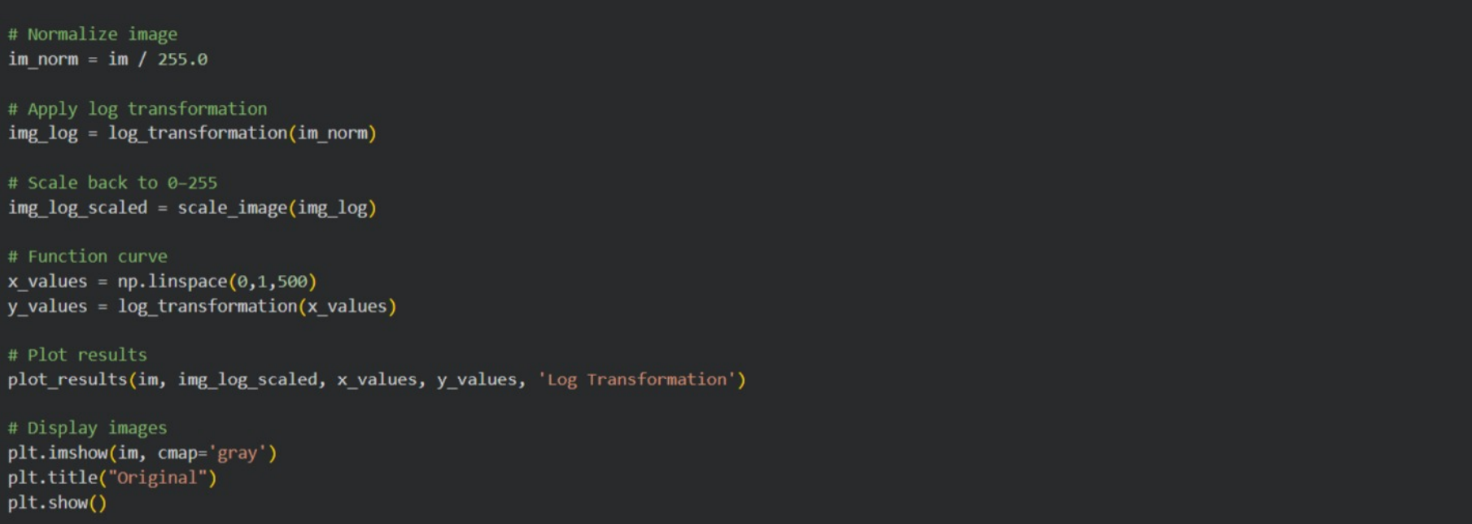
```
plt.imshow(img_neg)
```

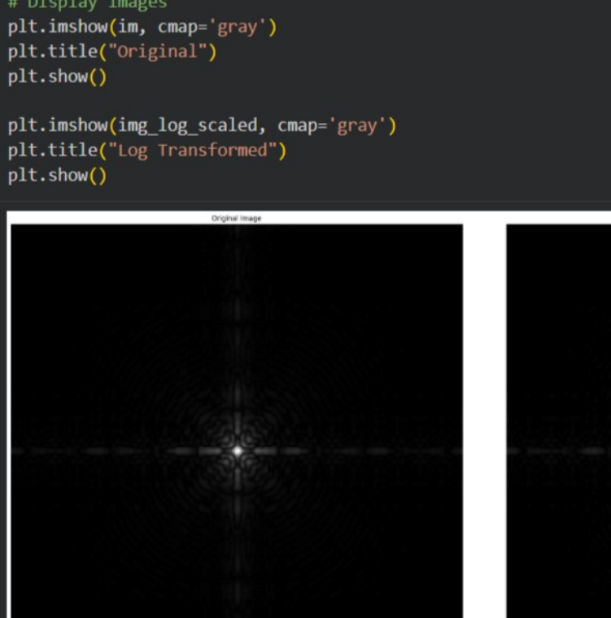
```
<matplotlib.image.AxesImage at 0x7a03a3dcd310>
```



Log transformation

```
import numpy as np
import matplotlib.pyplot as plt

def scale_image(input_img):
    input_img = input_img / np.max(input_img)
    input_img = (input_img * 255).astype(int)
    return input_img

def plot_results(input_img, output_img, x_values, y_values, title):
    plt.figure(figsize=(36, 12))

    plt.subplot(1,3,1)
    plt.imshow(input_img, cmap='gray')
    plt.title('Original Image')
    plt.axis('off')

    plt.subplot(1,3,3)
    plt.plot(x_values, y_values)
    plt.title(f'{title} Function')
    plt.xlabel('Input Intensity Levels')
    plt.ylabel('Output Intensity Levels')

    plt.subplot(1,3,2)
    plt.imshow(output_img, cmap='gray')
    plt.title(f'{title} Image')
    plt.axis('off')

    plt.show()

im = cv2.imread('Fig0305(a)(DFT_no_log).tif')

def log_transformation(r, c=1):
    s = c * np.log(1 + r)
    return s

# Normalize image
im_norm = im / 255.0

# Apply log transformation
img_log = log_transformation(im_norm)

# Scale back to 0-255
img_log_scaled = scale_image(img_log)

# Function curve
x_values = np.linspace(0,1,500)
y_values = log_transformation(x_values)

# Plot results
plot_results(im, img_log_scaled, x_values, y_values, 'Log Transformation')

# Display images
plt.imshow(im, cmap='gray')
plt.title("Original")
plt.show()

plt.imshow(img_log_scaled, cmap='gray')
plt.title("Log Transformed")
plt.show()
```



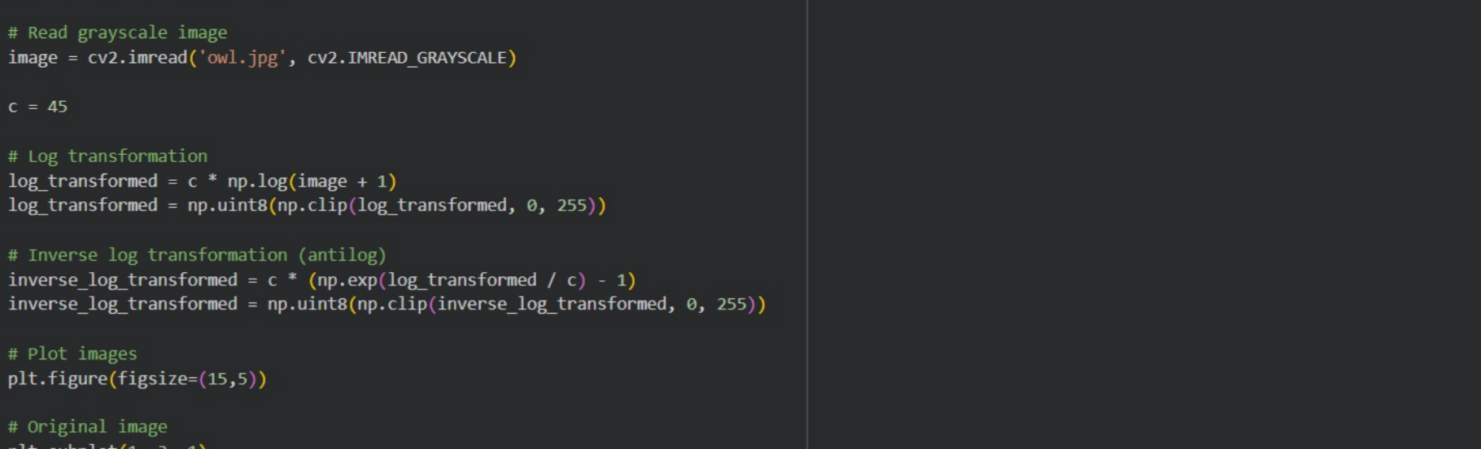


Log Transformed

Inverse Log Transformation

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read grayscale image
image = cv2.imread('owl.jpg', cv2.IMREAD_GRAYSCALE)

c = 45

# Log transformation
log_transformed = c * np.log(image + 1)
log_transformed = np.uint8(np.clip(log_transformed, 0, 255))

# Inverse log transformation (antilog)
inverse_log_transformed = c * (np.exp(log_transformed / c) - 1)
inverse_log_transformed = np.uint8(np.clip(inverse_log_transformed, 0, 255))

# Plot images
plt.figure(figsize=(15,5))

# Original image
plt.subplot(1, 3, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')

# Log transformed image
plt.subplot(1, 3, 2)
plt.imshow(log_transformed, cmap='gray')
plt.title('Log Transformed Image')
plt.axis('off')

# Inverse log transformed image
plt.subplot(1, 3, 3)
plt.imshow(inverse_log_transformed, cmap='gray')
plt.title('Inverse Log Transformed Image')
plt.axis('off')

plt.show()
```

```
/tmp/ipykernel_788/3950205030.py:11: RuntimeWarning: divide by zero encountered in log
  log_transformed = c * np.log(image + 1)
```

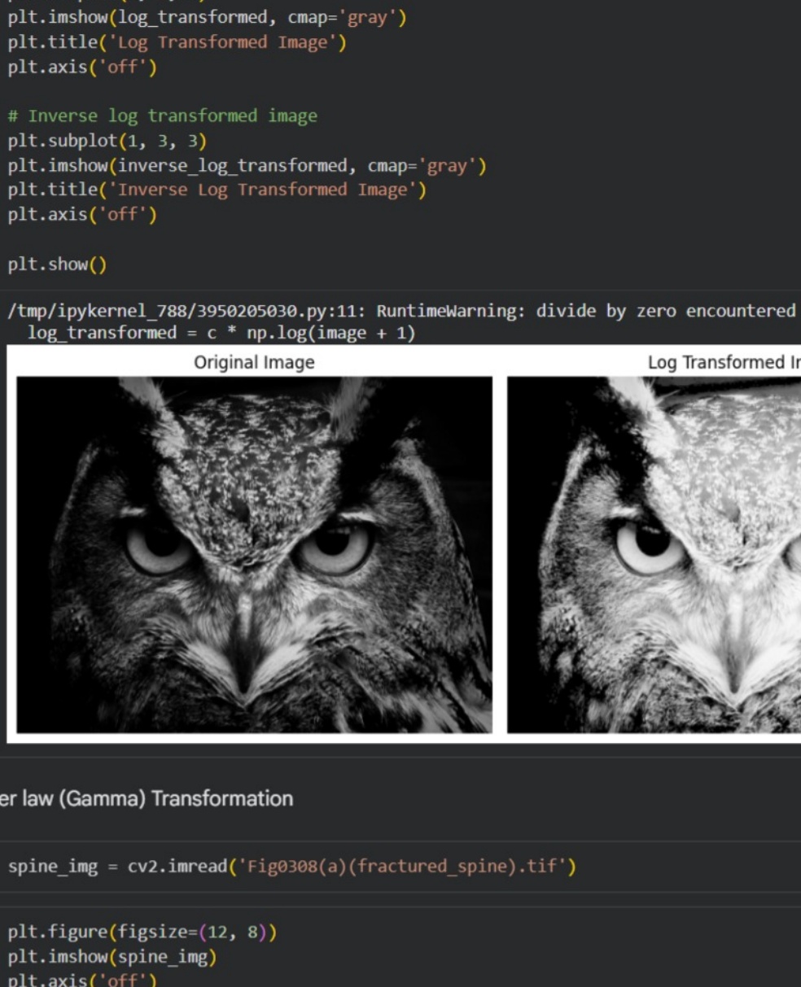


Power law (Gamma) Transformation

```
spine_img = cv2.imread('Fig0308(a)(fractured_spine).tif')
```

```
plt.figure(figsize=(12, 8))
plt.imshow(spine_img)
plt.axis('off')
```

```
(np.float64(-0.5), np.float64(745.5), np.float64(975.5), np.float64(-0.5))
```



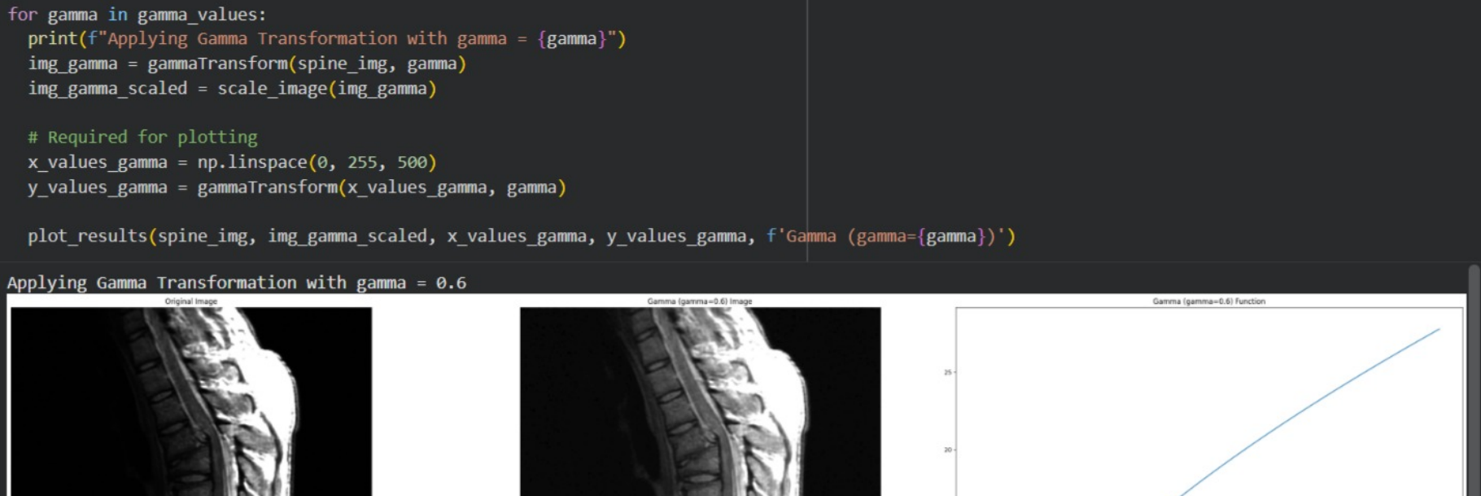
```
def gammaTransform(r, gamma, c=1):
  s = c * (r ** gamma)
  return s
```

Trying with gamma < 1

```
import numpy as np
import matplotlib.pyplot as plt

def scale_image(input_img):
    input_img = input_img / np.max(input_img)
    input_img = (input_img * 255).astype(int)
    return input_img
```

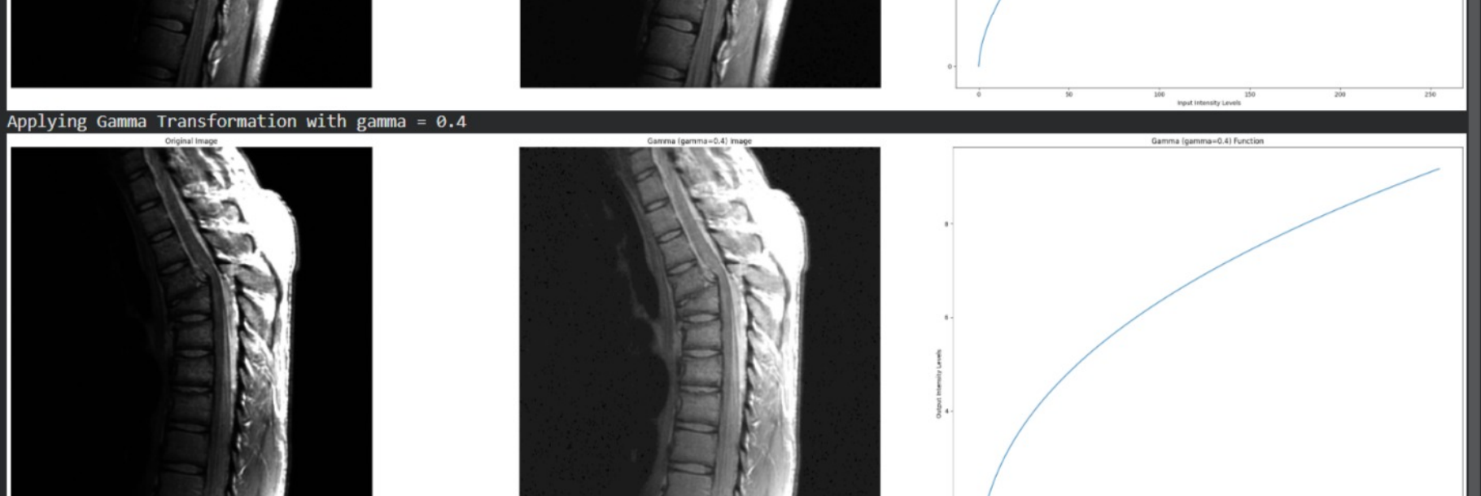
```
img_gamma = gammaTransform(spine_img, 0.4)
img_gamma_scaled = scale_image(img_gamma)
```

```
#require for plotting
x_values = np.linspace(0, 255, 500)
y_values = log_transformation(x_values, 0.4)

plot_results(spine_img, img_gamma_scaled, x_values, y_values, 'gamma_0.4')
```

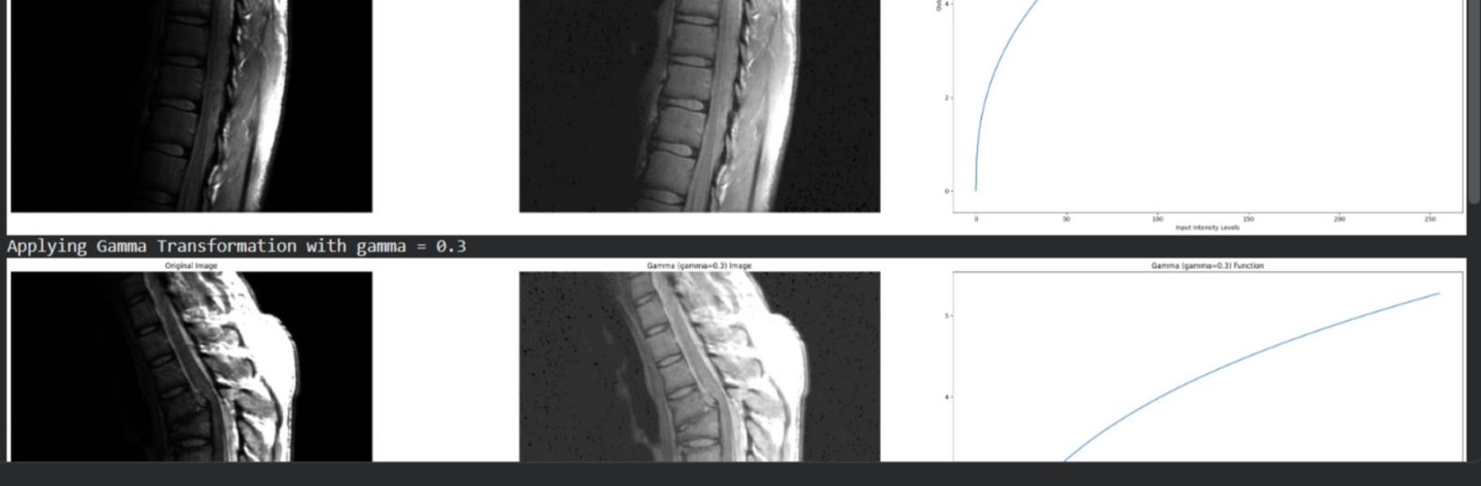


```
gamma_values = [0.6, 0.4, 0.3]

for gamma in gamma_values:
  print(f"Applying Gamma Transformation with gamma = {gamma}")
  img_gamma = gammaTransform(spine_img, gamma)
  img_gamma_scaled = scale_image(img_gamma)

  # Required for plotting
  x_values_gamma = np.linspace(0, 255, 500)
  y_values_gamma = gammaTransform(x_values_gamma, gamma)

  plot_results(spine_img, img_gamma_scaled, x_values_gamma, y_values_gamma, f'Gamma (gamma={gamma})')
```

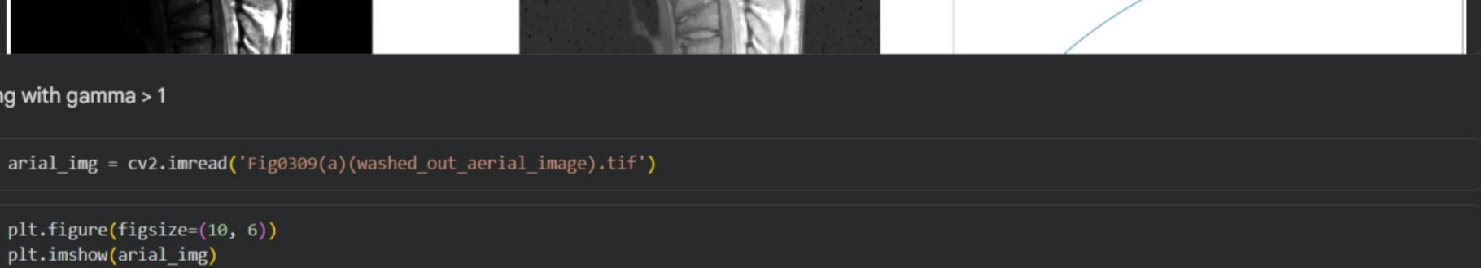
```
Applying Gamma Transformation with gamma = 0.6
```



```
Applying Gamma Transformation with gamma = 0.4
```



```
Applying Gamma Transformation with gamma = 0.3
```

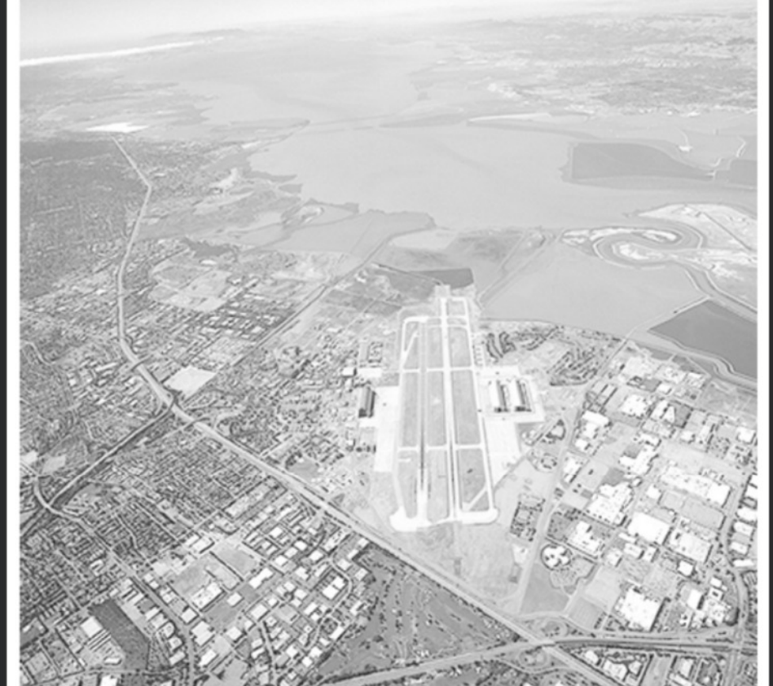


Trying with gamma > 1

```
arial_img = cv2.imread('Fig0309(a)(washed_out_aerial_image).tif')
```

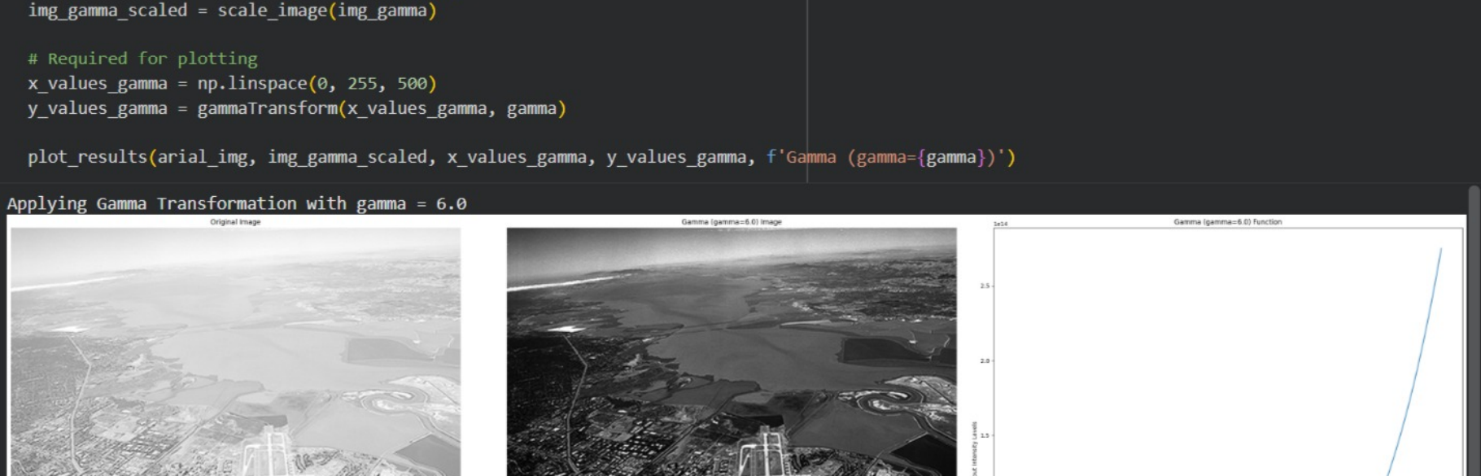
```
plt.figure(figsize=(10, 6))
plt.imshow(arial_img)
plt.axis('off')
```

```
(np.float64(-0.5), np.float64(764.5), np.float64(768.5), np.float64(-0.5))
```

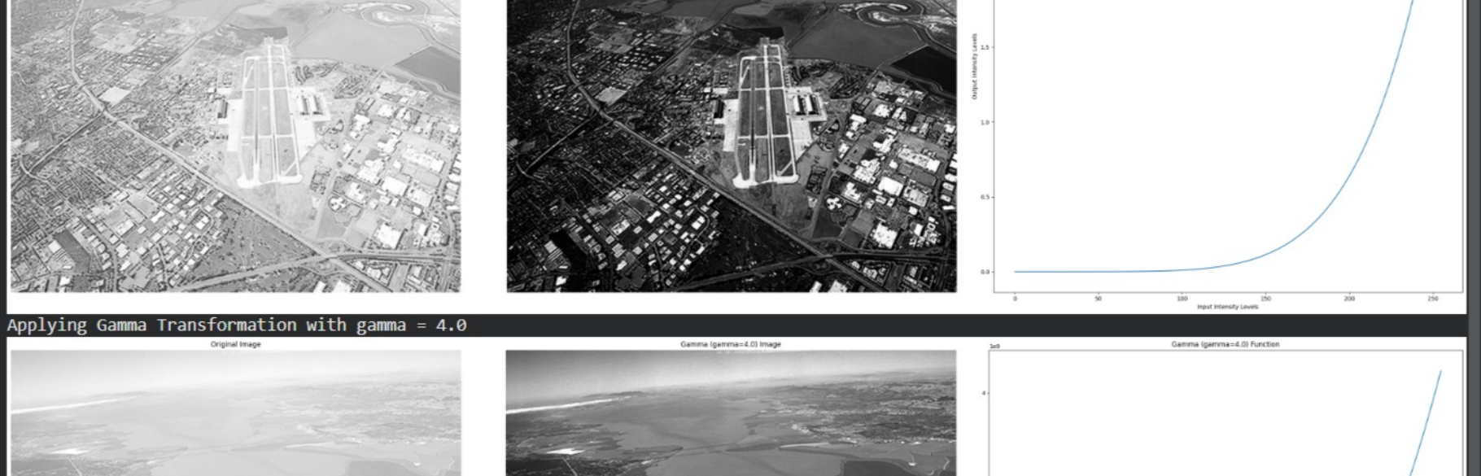


```
gamma_values = [6.0, 4.0, 3.0]

for gamma in gamma_values:
  print(f"Applying Gamma Transformation with gamma = {gamma}")
  img_gamma = gammaTransform(arial_img, gamma)
  img_gamma_scaled = scale_image(img_gamma)

  # Required for plotting
  x_values_gamma = np.linspace(0, 255, 500)
  y_values_gamma = gammaTransform(x_values_gamma, gamma)

  plot_results(arial_img, img_gamma_scaled, x_values_gamma, y_values_gamma, f'Gamma (gamma={gamma})')
```

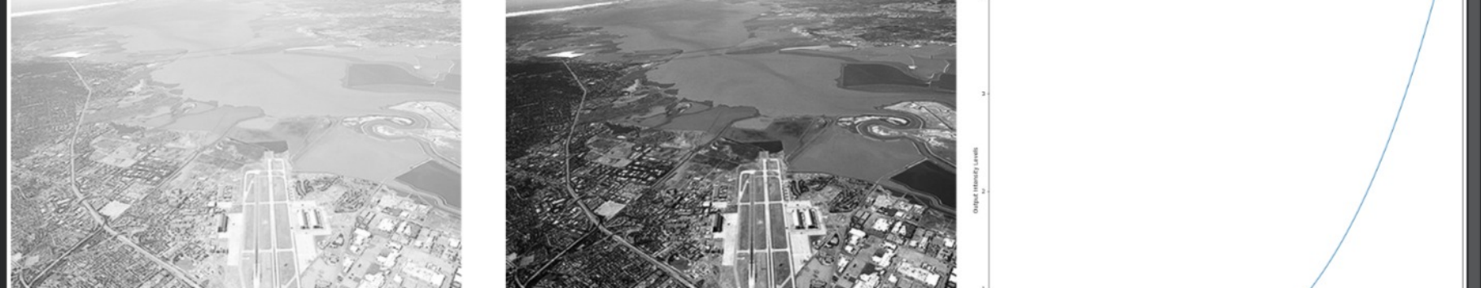
```
Applying Gamma Transformation with gamma = 6.0
```



```
Applying Gamma Transformation with gamma = 4.0
```



```
Applying Gamma Transformation with gamma = 3.0
```

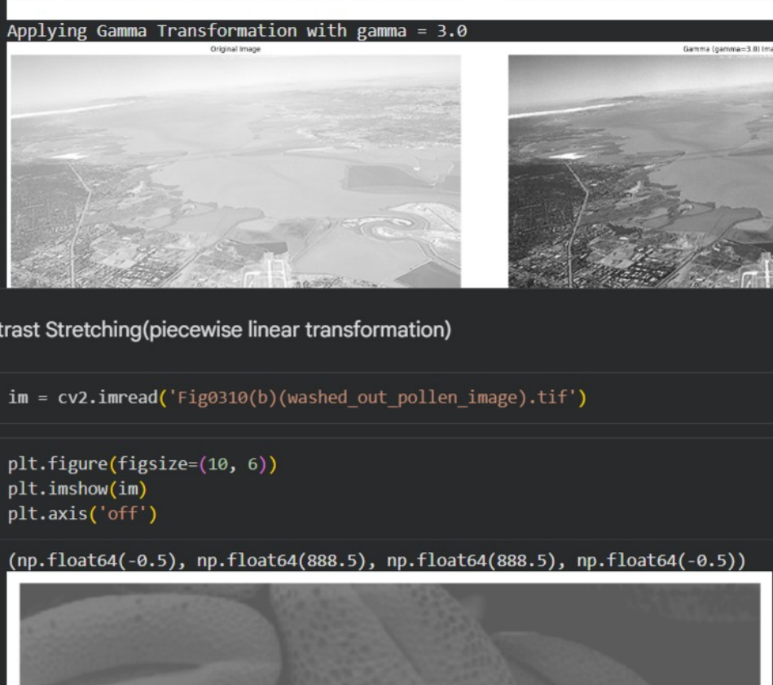


Contrast Stretching(piecewise linear transformation)

```
im = cv2.imread('Fig0310(b)(washed_out_pollen_image).tif')
```

```
plt.figure(figsize=(10, 6))
plt.imshow(im)
plt.axis('off')
```

```
(np.float64(-0.5), np.float64(888.5), np.float64(888.5), np.float64(-0.5))
```



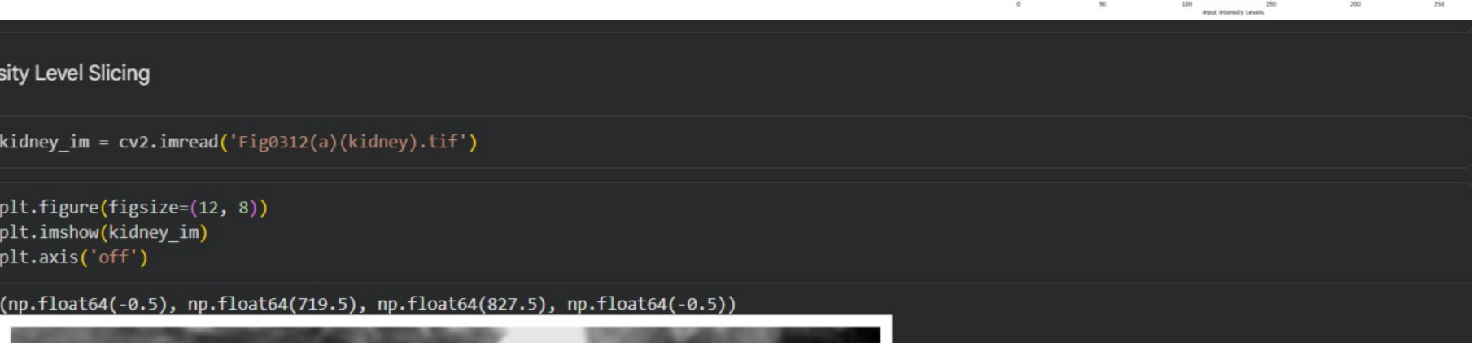
```
def piecewiseLinear(r, r1, s1, r2, s2):
  if r < r1:
    s = (s1 / r1) * r
  elif r > r1 and r < r2:
    s = ((s2 - s1) / (r2 - r1) * (r - r1) + s1)
  else:
    s = ((255 - s2) / (255 - r2) * (r1 - r2) + s2)

  return int(s)
```

```
piecewiseLinearVec = np.vectorize(piecewiseLinear)
```

```
x_values = np.linspace(0, 255, 500)
y_values = piecewiseLinearVec(x_values, 80, 30, 150, 190)
```

```
transformed_im = piecewiseLinearVec(im, 80, 30, 150, 190)
```

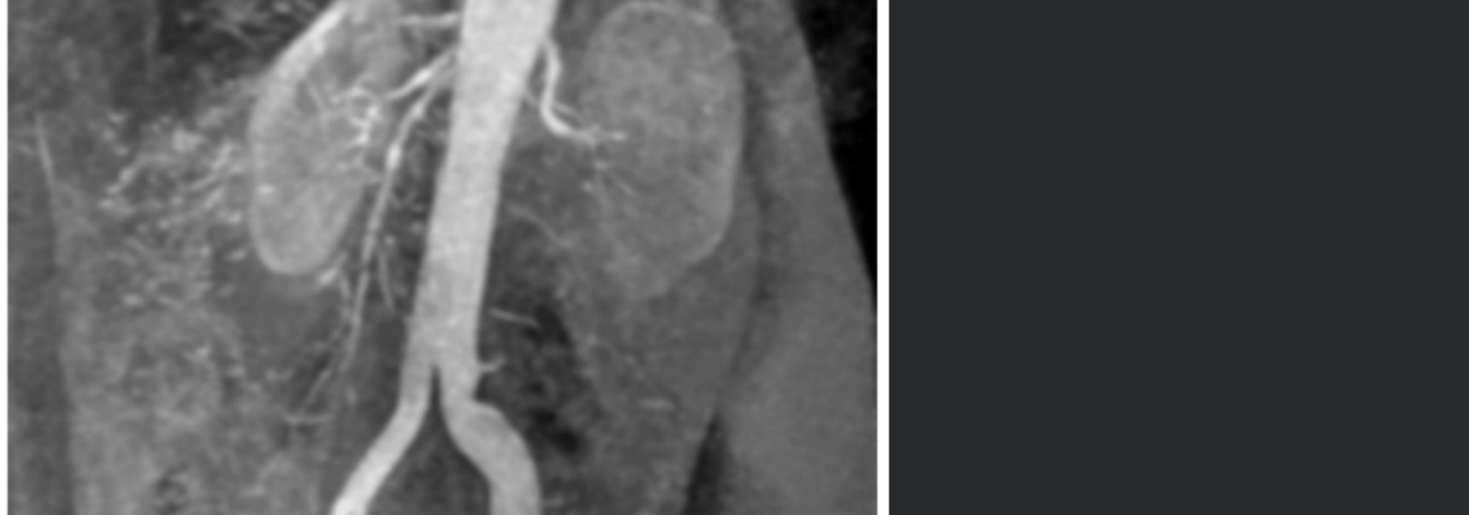
```
plot_results(im, transformed_im, x_values, y_values, 'piecewise1')
```



```
mean  = np.mean(im)
transformed_im2 = piecewiseLinearVec(im, mean, 0, mean, 255)
```

```
x_values = np.linspace(0, 255, 500)
y_values = piecewiseLinearVec(x_values, mean, 0, mean, 255)
```

```
plot_results(im, transformed_im2, x_values, y_values, 'piecewise2')
```

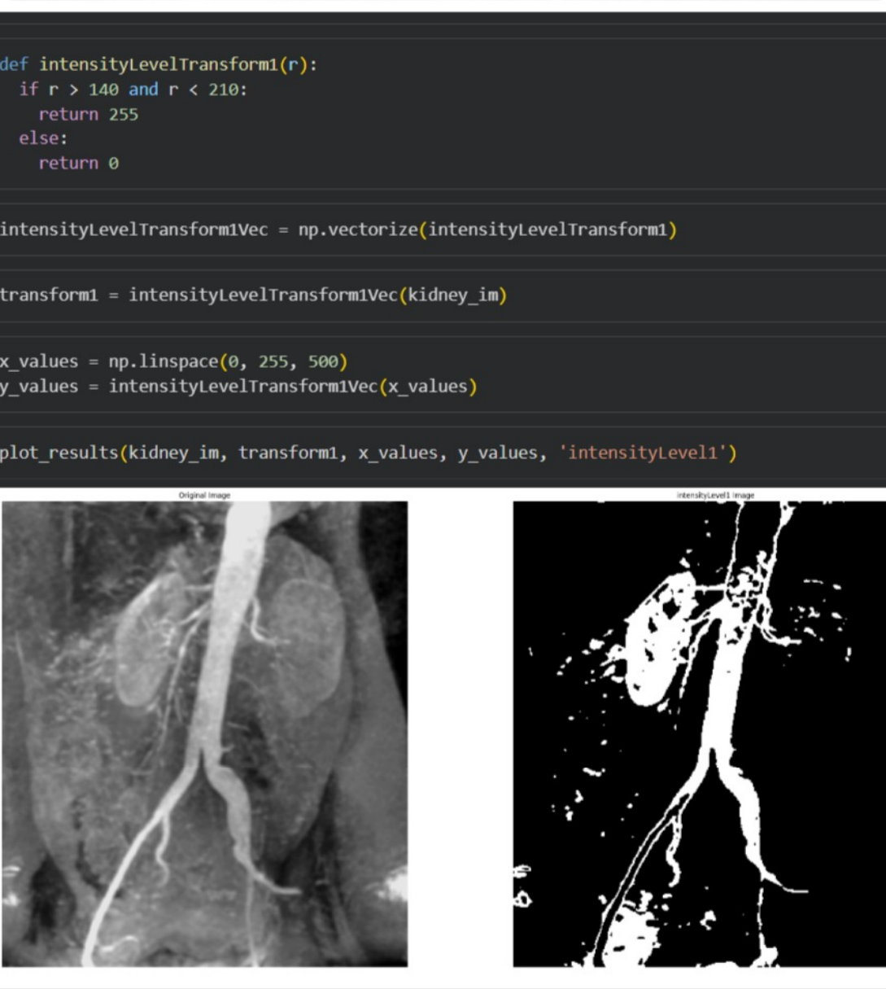


Intensity Level Slicing

```
kidney_im = cv2.imread('Fig0312(a)(kidney).tif')
```

```
plt.figure(figsize=(12, 8))
plt.imshow(kidney_im)
plt.axis('off')
```

```
(np.float64(-0.5), np.float64(719.5), np.float64(827.5), np.float64(-0.5))
```



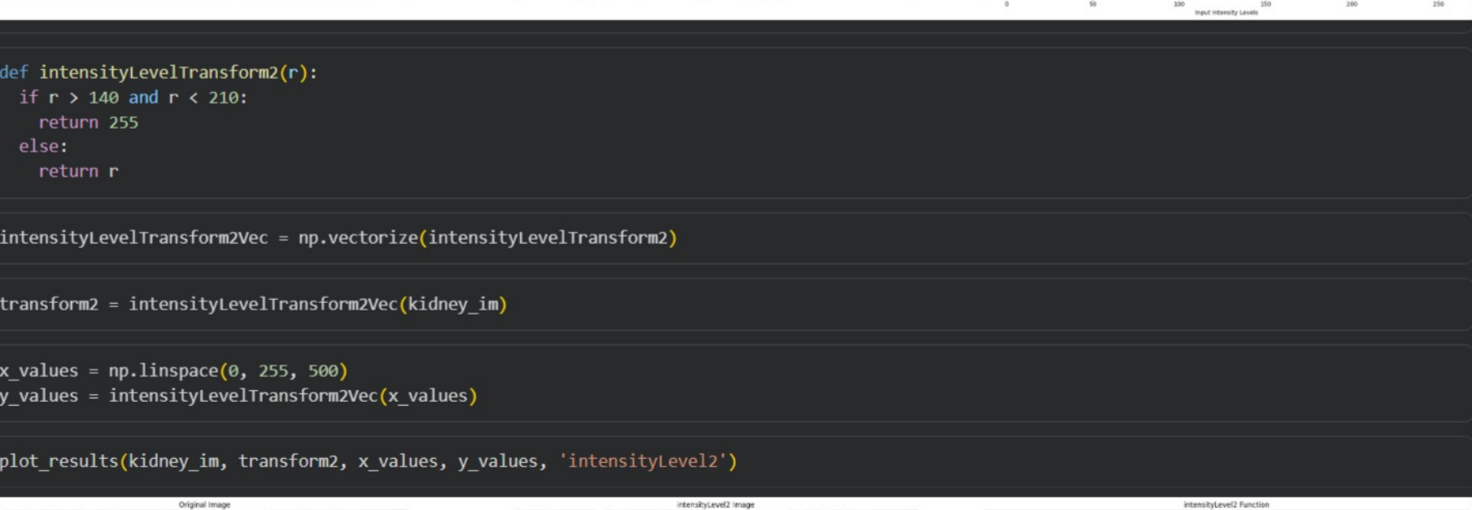
```
def intensityLevelTransform1(r):
  if r > 140 and r < 210:
    return 255
  else:
    return 0
```

```
intensityLevelTransform1Vec = np.vectorize(intensityLevelTransform1)
```

```
transform1 = intensityLevelTransform1Vec(kidney_im)
```

```
x_values = np.linspace(0, 255, 500)
y_values = intensityLevelTransform1Vec(x_values)
```

```
plot_results(kidney_im, transform1, x_values, y_values, 'intensityLevel1')
```



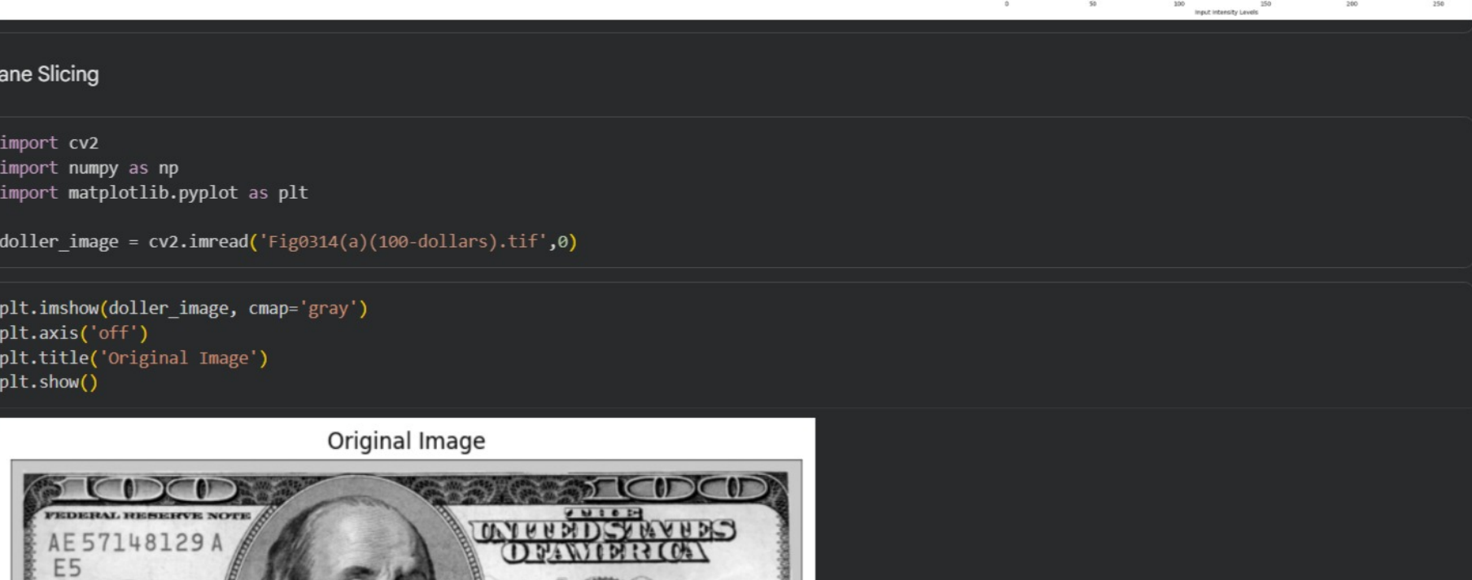
```
def intensityLevelTransform2(r):
  if r > 140 and r < 210:
    return 255
  else:
    return r
```

```
intensityLevelTransform2Vec = np.vectorize(intensityLevelTransform2)
```

```
transform2 = intensityLevelTransform2Vec(kidney_im)
```

```
x_values = np.linspace(0, 255, 500)
y_values = intensityLevelTransform2Vec(x_values)
```

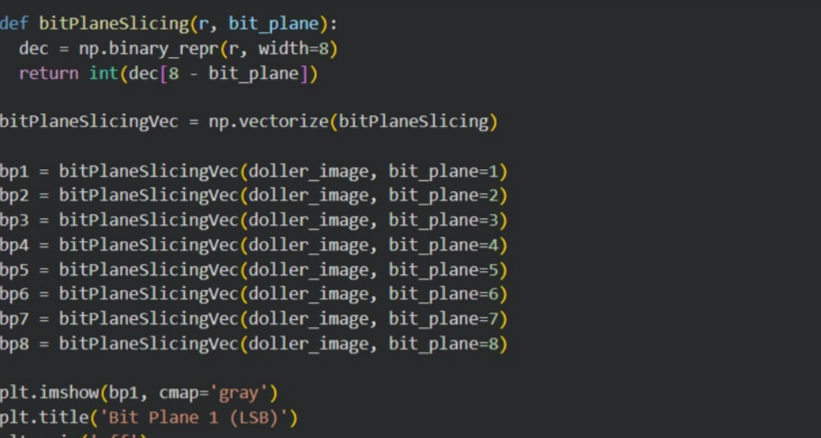
```
plot_results(kidney_im, transform2, x_values, y_values, 'intensityLevel2')
```

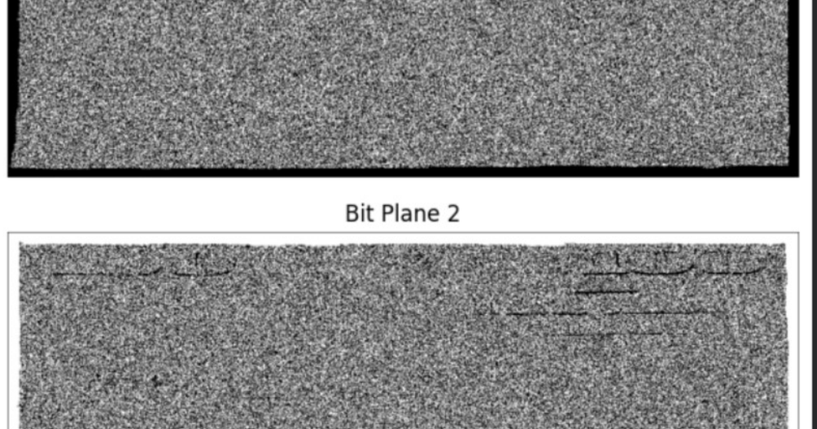
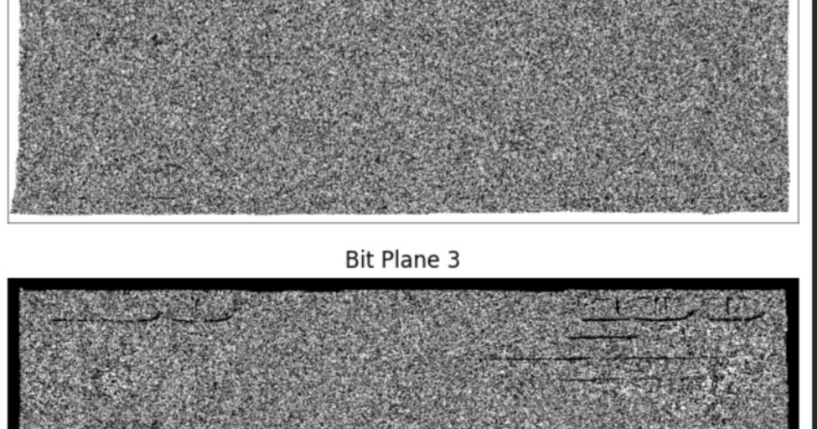
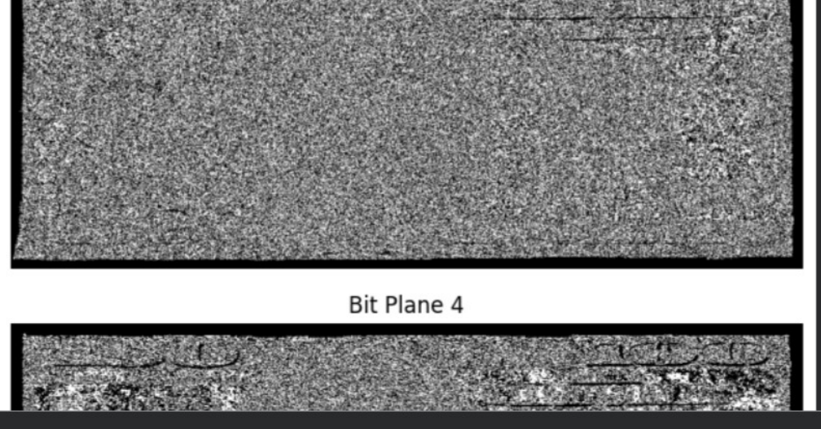


Bit Plane Slicing

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

doller_image = cv2.imread('Fig0314(a)(100-dollars).tif',0)
```

```
plt.imshow(doller_image, cmap='gray')
plt.axis('off')
plt.title('Original Image')
plt.show()
```



```
def bitPlaneSlicing(r, bit_plane):
  dec = np.binary_repr(r, width=8)
  return int(dec[8 - bit_plane])

bitPlaneSlicingVec = np.vectorize(bitPlaneSlicing)

bp1 = bitPlaneSlicingVec(doller_image, bit_plane=1)
bp2 = bitPlaneSlicingVec(doller_image, bit_plane=2)
bp3 = bitPlaneSlicingVec(doller_image, bit_plane=3)
bp4 = bitPlaneSlicingVec(doller_image, bit_plane=4)
bp5 = bitPlaneSlicingVec(doller_image, bit_plane=5)
bp6 = bitPlaneSlicingVec(doller_image, bit_plane=6)
bp7 = bitPlaneSlicingVec(doller_image, bit_plane=7)
bp8 = bitPlaneSlicingVec(doller_image, bit_plane=8)

plt.imshow(bp1, cmap='gray')
plt.title('Bit Plane 1 (LSB)')
plt.axis('off')
plt.show()

plt.imshow(bp2, cmap='gray')
plt.title('Bit Plane 2')
plt.axis('off')
plt.show()

plt.imshow(bp3, cmap='gray')
plt.title('Bit Plane 3')
plt.axis('off')
plt.show()

plt.imshow(bp4, cmap='gray')
plt.title('Bit Plane 4')
plt.axis('off')
plt.show()

plt.imshow(bp5, cmap='gray')
plt.title('Bit Plane 5')
plt.axis('off')
plt.show()

plt.imshow(bp6, cmap='gray')
plt.title('Bit Plane 6')
plt.axis('off')
plt.show()

plt.imshow(bp7, cmap='gray')
plt.title('Bit Plane 7')
plt.axis('off')
plt.show()

plt.imshow(bp8, cmap='gray')
plt.title('Bit Plane 8 (MSB)')
plt.axis('off')
plt.show()
```